

LivePremier Plus user guide

LivePremier Plus is a **local app that puts extra panels inside an Analog Way LivePremier Web RCS session**, drawn in Web RCS's own design language so they read as part of the product rather than as a bolt-on.

- **Edit** — the Screens / Aux. page with one row instead of two, and that row is a *programmer*: a buffer that is on neither preview nor program. Build a look with the same sources, the same layer parameters and the same memory bank, and the switcher sees none of it until you save it into a memory.
- **VPU Map** — the device's mixing-resource allocation, drawn as a budget. Which units are fitted, who holds them, what is spare, and what a staged preconfig would change. The link grid reads top to bottom the way an output link runs: a screen's native layer in a band above the eight layer links, the layers down the field, and — when a screen ran out of mixers and its next layer is on another VPU — on into that VPU's card, stacked underneath. The header over the columns names each link's screen, and its region and output plug where the outputs add up to the screen's own figures.
- **Console** — a lighting-desk command grammar for a video switcher.
- **Timeline** — a theatre-style cue stack that advances on one GO, with per-cue fade, delay and follow times.
- **Layer Groups** — several layers, on one screen or on many, driven as one; a ganged group follows a source change made to any member, wherever it came from.
- **Send to** — a  on every source card that routes it to a screen and layer, or to a whole group, in preview or program, without a drag.
- **Matrix Routing** — patch the frame's own SDI and HDMI sockets to ports on a Blackmagic Videohub, a Lightware or a Turtle AV router, and route through it from the panel, a cue, the Console or OSC.
- **MIDI Mapping** — a control surface driving the switcher, from the page itself.
- **Arithmetic in the vendor's own numeric fields** — type `1080-80` into a layer width and get 1000.

It rides the vendor app's own WebSocket. **No second connection to the device, no replacement UI, and nothing to install in the browser.**

Before you rely on this: the panels render inside a real Web RCS session and the device store mirrors live — verified through this proxy against **LivePremier Simulator 6.2.73**, along with cue-stack persistence and the whole setup flow. The VPU map has been **read from a live Aquilon C** and is tested against that capture, including Optimized mode, interleaved output links, and a staged preconfig differing from the running one.

Midra 4K and Alta 4K (QuickVu, Pulse, Eikos, QuickMatrix, Zenith 100/200) are supported since 0.6.0 — Timeline, Console, Memories, Layer and Pitch Compensation; there is no VPU to map on those and MIDI Mapping is not offered there yet. The port was proven on a **live Pulse 4K**: every write over the device protocol by a test harness, and the Console panel's own lines typed by the operator — the first writes this app's panels have made to real hardware. On LivePremier, nothing has been written to a physical device yet and the timeline has only ever fired at a simulator.

Built with AI assistance, directed and reviewed by a human author.

Running it

Needs **Node 20 or newer** and has no dependencies to install.

```
npm start
```

Then open `http://127.0.0.1:8535/` and give it the switcher's address. That address is remembered.

```
npm start -- --device 192.168.2.142
```

flag	meaning
<code>--device <host[:port]></code>	the switcher. Port defaults to 80 (a simulator is usually <code>:3000</code>).
<code>--port <n></code>	local port to listen on (default 8535)
<code>--host <addr></code>	local address to bind (default <code>127.0.0.1</code>). Bind a LAN address or <code>0.0.0.0</code> to reach it from other machines. On the machine running it, keep using <code>http://127.0.0.1:<port>/</code> — that is the address browsers treat as secure, which MIDI Mapping and the timecode sources need; a local browser that arrives by the LAN address is sent there automatically. Other machines get everything except those.
<code>--data <dir></code>	where cue stacks are kept

There is a desktop app too — a tray launcher with an interface and port picker.

On binding wide. The default is loopback for a reason: **this proxy is an unauthenticated route to a switcher's entire control surface.** `--host 0.0.0.0` hands that to everyone on the network. Do it deliberately, not by habit.

Open it through the proxy, not at the switcher

If you open the switcher's own address directly, the panels are not there and MIDI will not work.

Web MIDI is a secure-context API; `http://127.0.0.1:<port>` counts as one and a plain-HTTP LAN address does not. The panel says so rather than failing silently.

Edit — programming a look off the buses

PLUS ▶ Edit. The Screens / Aux. page, with one row per destination instead of two. Sources on the left, the destinations in the middle, the layer panel on the right — the vendor's own layout, so everything is where you already look for it. The difference is the row: it is marked **EDIT** in amber rather than PGM in red or PRW in green, because it is on neither bus.

Nothing you do here reaches the switcher. There is no TAKE, no T-bar and no padlock, because there is nothing to transition and nothing to protect.

Building a look

Each card starts empty. Three ways to fill it, and none of them asks the device for anything:

From PGM	copy what is on air into the programmer
From PRW	copy what is cued
Empty	every layer full-frame with no source, which is what an untouched preset looks like

Then work the way you would on the real page:

- **click a layer** on the card, or a row in the strip beneath it, to select it;
- **drag it** to move, or take one of the eight handles to resize — the numbers land in the Layer panel as you go, clamped to the switcher's own limits;
- **click a source** on the left to put it on the selected layer;
- **Layer** on the right is the full parameter set — all sixty-seven of them, the same panel the vendor's tab strip carries, pointed at the programmer.

Discard forgets a destination's buffer entirely; **Empty** keeps it and clears it.

Saving it into a memory

The **Memory** tab is the one control on this page that touches the switcher. Name a slot, give it a label, and choose a route:

- **Direct** — writes the memory bank itself. No preset buffer is written and no take is fired, so neither preview nor program moves. This needs the switcher to be able to read a file this app writes, which it can when both are on the same machine — a simulator, or an installation where `memoryImportDir` points at a share they both see.
- **Via preview** — puts the look into the preview buffer, fires the switcher's own save, then puts preview back property-for-property. **Program never moves.** Preview shows the look for about a second, and afterwards the bank will call it *modified* even though the content is identical — the switcher is right that the buffer was written to. Refused while a take is in flight. This route works on any switcher, and it is the only one on Midra 4K and Alta 4K.

Load into the programmer reads a slot back the other way. A memory's contents are not in the device's own data — a slot only publishes its name, its canvas and what categories it holds — so reading it out is the only way to see inside one, and it is how you pick up yesterday's look and change it.

Console

Verb first, then objects, innermost scope last. Every keyword abbreviates to any unambiguous prefix; **Enter executes and nothing is sent before it.**

```
Recall Screen 1 Memory 5
R Sc 1 Th 4 Me 5 Pre      the same command
Store Master 12
Take Screen 1
```

The line **parses as you type and shows what it will do** — `Recall 3 → Screen 1 Preview` — before anything reaches the device. Tab completes; ↑ recalls history.

This panel owns no grammar. Every token, rule and device path comes from `mynah`'s own build output — the same artefact its Companion module vendors. **If a command means the wrong thing, the fix is in mynah.**

That is worth more than tidiness: `mynah`'s compiler and this repo's command builder were arrived at independently, and they emit **byte-identical** store paths for the commands both know. Two independent derivations agreeing is the strongest evidence either is right, and it stays true only while nobody re-types the grammar here.

Layer Groups, and the `...` on a source card

PLUS ▶ Layer Groups. A group is a name and a list of layers, on one screen or on several — layer 2 on screen 1 with layer 1 on screens 2 and 3 is *the side screens*. Add a layer with the destination and

layer pickers at the bottom of each group card; the layer list is the device's own **fitted** list, so a slot the hardware has not got is never offered.

The two source columns beside each member show what it is showing now, in program and in preview, so a glance says whether the group agrees.

Gang: follows is on by default. The rest of the group is written to match whenever any member's source changes — by the  menu, by the vendor's own drag-and-drop, by a memory recall, by another client. Turn it off and the group stays useful as a target and does nothing on its own.

A layer belongs to **one group at a time**; adding it somewhere takes it out of where it was, and the panel says so.

Sending an input

Every source card in the Sources panel gets a  beside the vendor's own . The  is theirs and opens that input's settings; the  is ours and routes it.

```
Send IN4
[ Preview ] [ Program ]
Recent      Side screens      S1 L2 · S2 L1  GANG
Groups      Side screens      S1 L2 · S2 L1  GANG
Screens     S1  [L1] [L2]     S2  [L1]
```

Preview and program mean the **role**, resolved per screen at the moment you click. Two screens sitting on opposite preset letters — which is normal — still both get it in the right buffer.

The PGM padlock is respected. With program unlocked on the screens involved, a program send goes through on one click. With it locked, you get a step naming every layer it is about to change and asking. A screen you have not got on the page has no padlock to read, so it is treated as locked and asked about too.

Mid-take nothing is sent. While a transition is in flight, "program" and "preview" do not name a buffer honestly, and the menu says so rather than guessing.

Groups are kept **per switcher**, beside the cue stack. Point the app at a different frame and you get that frame's groups.

Matrix Routing

PLUS ▶ Matrix Routing. Patch the switcher's own sockets to ports on an external router — a **Blackmagic Videohub**, a **Lightware**, or a **Turtle AV** — and route through it from here.

You describe the **cable**, and the rest follows. The direction inverts, which is the thing that reads wrong at first glance every time:

```
switcher INPUT  <--- cable ---  router OUTPUT  (the router feeds us)
switcher OUTPUT --- cable --->  router INPUT    (we feed the router)
```

So a switcher input is fed by a router *output*, and a switcher output arrives at a router *input*. The patch form says which of the two it wants as you fill it in.

Set it up in three steps. Add the router (name, protocol, address — the port follows the protocol's own default). Patch each cable: pick a socket, pick the router, give the port number at the other end. Then route.

Sockets are named in the device's own words, because which connector "card 2, port 1" is depends on a legend this app cannot see:

```
Input 13 · card IN_2 · connector IN_25 · sdi
Output 5 · card OUT_2 · hdmi
```

The two route controls are different, and deliberately so. An input row gets a source dropdown — one choice, because a socket is fed by one router output, and it settles what that input sees. An output row gets a destination field, because the signal arrives at one router input and can go to any number of outputs at once: type `3`, or `1-4`, or `1,2,5-8`.

⚠ Sending adds; it never takes away. Naming outputs 1-4 routes those four and leaves output 5 alone, even if it was showing this source a moment ago. A router output always shows *something*, so "removing" a destination would mean choosing a different source for it — and there is no answer to which one.

The Now column is what the router says, not what you asked for. A click does not move the grid until the router agrees, and it keeps up with changes made at the router's own front panel or by another operator. A Videohub answers a route it will not make with an acknowledgement and the unchanged crosspoints, so a panel that showed your request back to you would look right and be wrong — during exactly the minute that matters.

From a cue, the Console and OSC

A cue can carry a matrix route beside its recalls and takes. It goes out ahead of the take, so the signal is there before anything switches to it.

⚠ A cue does not wait for the signal to lock. An SDI reclock is quick; an HDMI or HDCP handshake through a router can take a second or more, and nothing reports when it is done. A cue that routes and takes in one breath can take to black — put the route in an earlier cue when the format may change.

The same three addresses work typed at the Console and sent over OSC:

```
/lp/matrix/input/5/source      7      switcher input 5 now sees router input 7
/lp/matrix/output/2/destinations "1-4"  switcher output 2 out of router outputs 1-4
/lp/matrix/hub/route/3        9      router "hub": output 3 takes input 9
```

The routers are kept for the **installation**, not per switcher — a Videohub does not move when you fail over to a backup frame. The **patch** is per switcher, beside the cue stack, because it describes that frame's own sockets.

What has been proven, and what has not. The Videohub driver was driven end to end against a working implementation of the protocol, including the crosspoint numbering, which counts from zero on the wire and from one everywhere you can see. **No real Videohub has been in the loop.** The Lightware and Turtle AV drivers are written from their vendors' protocol documents and **have never spoken to the hardware at all.** docs/MATRIX.md gives a short procedure for proving each on your own kit before a show.

MIDI Mapping

Under **Virtual RC400T**. Pick an input, an output for feedback, and a controller profile; press Start. Faders, encoders and buttons then drive the selected layer.

The engine is [awj-surface](#)'s, vendored whole along with its stock profiles — X-Touch/Mackie, APC40, MIDIcon 2 and Pro, plus a generic learn profile.

Soft pickup is the behaviour worth knowing. A non-motorised fader will not move a live value until it has swept *through* that value, so picking up a fader mid-show cannot jump a layer's opacity. **The panel shows the hold-off rather than looking broken** — if a fader appears dead, that is what you are seeing.

Arithmetic in numeric fields

Type an expression into one of Web RCS's own numeric fields and it is evaluated when you commit — on Enter, or on leaving the field:

typed	becomes
1080-80	1000
(1920-40)/2	940
1920*2	3840

+ - * / () and decimals, with the usual precedence. **The result is clamped to the field's own declared range and rounded to its step**, so a width of 9000+1000 lands on 8192 rather than being

refused.

Nothing else about the field changes: the vendor still validates it, still decides what to send, and still drags the other axis along if the aspect lock is on.

Where it applies is narrow on purpose. Only fields the vendor itself marks as numeric by putting `min`, `max` and `step` on them. That excludes everything that must not be touched — labels have no `step`, and neither does the transition-time field, whose `00:01.000` would be mangled by anything treating `:` as arithmetic.

Opacity and zoom are deliberately excluded. They are real number inputs, and a number input *discards* anything it cannot parse — after typing `1080-80` the browser reports an empty value, and the expression is visible on screen but unreadable from script. Winning arithmetic there would mean mutating a vendor element's type on every focus, in a UI driving a live show. Not a good trade.

There is **no** `eval` anywhere in this path; anything the parser does not fully understand is refused rather than guessed at.

The demo environment

```
npm run demo
```

brings the whole thing up against a **LivePremier Simulator**, with a real Aquilon C's resource map folded in and an example cue stack loaded, so every panel has something real to show without a switcher in the room.

The simulator has **no VPU at all**, so the panel against a bare one correctly reports there is nothing to draw; the demo supplies that from the recorded capture — 32 of 64 mixers fitted, 26 allocated, one screen in Optimized mode, and a staged preconfig differing in 26 mixers. Everything outside that stays the simulator's own live state, **so cues fired from the timeline really do go on the wire**.

It refuses to run against anything that is not loopback. The demo splices a store subtree and seeds a cue stack, and "I thought it was the simulator" is exactly the mistake worth making impossible.

The demo's cue stack lives in a temporary directory and is deleted on exit; your own is never touched.

Saving and restoring your setup

A `.awc` from the switcher restores the *processor* — inputs, outputs, screens, memories. It holds nothing about this app: your cue stack, your layer groups, the names you gave layers, the patch to the external routers. Restore a `.awc` on its own onto a fresh frame and the show comes back with no cue list.

So this app can write all of that into one file:

```
curl -o livepremier-plus.json 'http://127.0.0.1:8535/___lpp/config?download=1'
```

It is plain JSON — readable, diffable, fine in a git repo. It carries which switcher it was written against, and three separate groups:

- **installation** — app settings (console language, OSC port and bind) and your external routers. Not tied to a switcher.
- **show** — the cue stack, layer groups and layer names. Tied to the switcher they were built on, because they name screens and layer slots like `S1/2`.
- **rig** — the patch between the frame and the routers.

To restore, POST it back:

```
curl -X POST -H 'content-type: application/json' \
  --data-binary @livepremier-plus.json \
  http://127.0.0.1:8535/___lpp/config
```

By default that restores the show and the rig and your routers, and **leaves the app settings alone** — a restore should not quietly change the OSC port a lighting desk is sending to. Ask for them explicitly if you want them:

```
{ "doc": { ... }, "sections": ["stack", "groups", "names", "patch", "settings"] }
```

Onto a different frame. The device-keyed parts land under whichever switcher this app is currently pointed at, so failing over to a backup frame at another address is just: point the app at the backup, then POST the file. Add `"device": "192.168.2.141"` to send it somewhere else again. To see what a file would do before doing it, POST it to `/___lpp/config/inspect`.

Both halves in one file. `Showbook` keeps this file beside the `.awc` for the same show and exports the pair as one `.showbook` bundle — or, if you want a single file you can restore straight from the switcher's own Web RCS, it can put this configuration *inside* the `.awc`. The switcher accepts it and ignores it; this app reads it back out. Note the switcher does not keep it: a `.awc` you export from Web RCS afterwards will not contain it.

If something is wrong

Symptom	Cause
No panels in Web RCS	You opened the switcher's address directly. Go through the proxy.
MIDI does nothing	Same cause — Web MIDI needs a secure context, which loopback is and a LAN address is not.
A fader does not move anything	Soft pickup. Sweep it through the current value.
A console command means the wrong thing	The grammar is mynah's; the fix is there.
Arithmetic does not work in a field	It is not one the vendor marks numeric — opacity and zoom are deliberately excluded.
The VPU panel says there is nothing to draw	A simulator has no VPU. That is correct, not a failure.
The demo refuses to start	It only runs against loopback, on purpose.